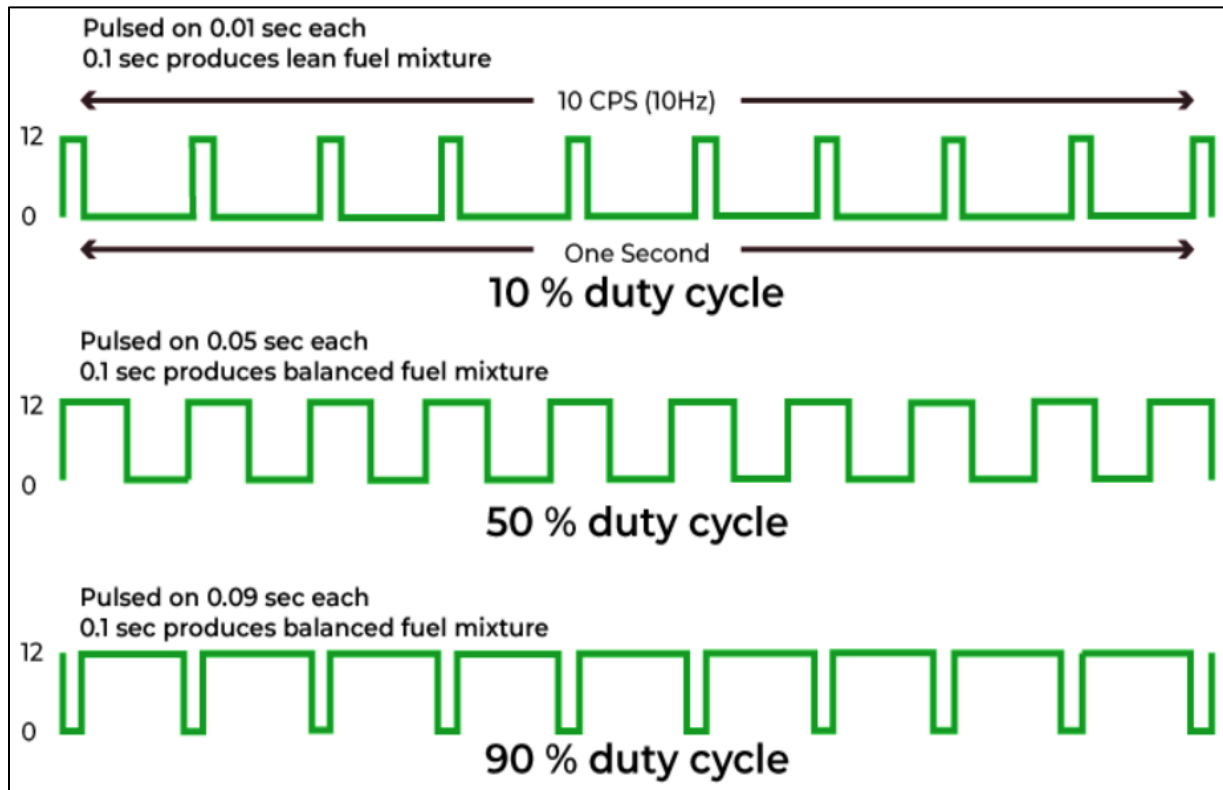


PWM 8BIT → 500HZ



Professor: Dr. Miguel Angel Rivera Acosta

Students:

Alonso Emmanuel López Macias

Cesar Eduardo Inda Cenicerros.

Luis Alberto Castañeda Montaña

José Antonio Garcia Madrigal

Team:9

Date: June 20th, 2025

THEORETICAL FRAMEWORK

1.- INTRODUCTION

The objective of this project is to **design and simulate an 8-bit PWM (Pulse Width Modulation) signal generator using the System Verilog hardware description language and the Vivado development tool. The design is carried out at the RTL (Register Transfer Level)**, without the need to implement the circuit on a physical FPGA, focusing solely on its functional modeling and verification through simulation

2.- DEFINITION

PWM (Pulse Width Modulation) is a technique used to generate digital signals with a periodic waveform, where the duty cycle of the signal is controlled. The duty cycle defines the proportion of time the signal remains at a high level ('1') relative to the total period. Common applications of PWM include:

- *Speed control of DC motors*
- *Brightness regulation of LEDs*
- *Generation of analog signals through filtering*
- *Data communication in certain protocols*

A PWM signal does not change its frequency but varies its pulse width to represent different levels of power or intensity.

3.- FUNDAMENTALS

The PWM generator to be developed in this project is defined with the following characteristics:

- **Resolution:** 8 bits (256 possible duty cycle levels).
- **Frequency:** 500 Hz.
- **Control input:** an 8-bit value that defines the duty cycle.

To generate the PWM signal, a counter is used that increments from 0 to 255 periodically. The duty cycle value is compared with the counter value in each cycle. If the counter is less than the duty cycle value, the PWM output will be high; otherwise, it will be low. Example: **if the control value is 128, the duty cycle will be 50% (128/255)**. The general formula for calculating the duty cycle is:

$$DutyCycle (\%) = \frac{Control Value}{255} \times 100$$

Equation 1 Pwm DutyCycle

To generate a 500 Hz signal with 256 steps per period, each step must occur every:

$$\frac{1}{\frac{500}{256}} = 7.8125 \mu s$$

Equation 2 256Steps --> 500 Hz

The consideration for **PWM Period** is:

$$TPWM = \frac{1}{f_{PWM}} = \frac{1}{500Hz} = 2ms$$

Equation 3 PWM Period

The consideration for **Hardware Clock Division** is:

$$f_{CLK} = 50MHz, N = 256, f_{PWM} = 500Hz$$

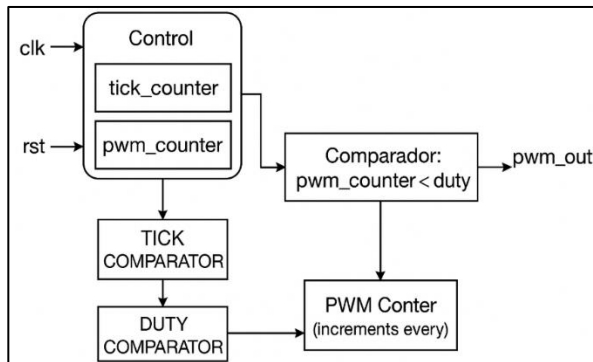
$$Cycles By Pass = \frac{f_{CLK}}{f_{PWM} \cdot N} = 390 Cycles$$

Equation 4 Cycle By Pass FPGA

These equations represent the conditions for any consideration in the times or frequency requirements.

ESPECIFICATIONS

4.- MICROARCHITECTURE



clk: Reloj del sistema.

rst: Reset síncrono, activo en alto.

duty_cycle[7:0]: Valor del ciclo de trabajo (0 = 0%, 255 = 100%).

pwm_out: Señal PWM generada (digital: 0 o 1).

Tick Counter (tick_counter). - Cuenta cuántos ciclos de reloj han pasado. Se reinicia cuando alcanza TICKS_PER_STEP.

PWM Counter (pwm_counter). - Cuenta de 0 a 255 y se incrementa cada vez que tick_counter reinicia.

Comparador PWM. - Compara $pwm_counter < duty_cycle$. Si es verdadero, $pwm_out = 1$, de lo contrario 0.

Generador PWM. - Resultado del comparador se envía a pwm_out , generando la onda cuadrada modulada.

NOTAS:

TICKS_PER_STEP define la resolución temporal para lograr 500 Hz.

PWM_RESOLUTION = 256 define los niveles de precisión de duty.

Esta arquitectura genera una frecuencia constante con un duty variable.

5.- OPERATION SEQUENCE

1. Al comenzar, el sistema se **resetea** y los contadores se ponen en 0.
2. En cada flanco de reloj (**posedge clk**), el **tick_counter** se incrementa.
3. Si **tick_counter == TICKS_PER_STEP - 1**:
 - Se reinicia tick_counter.
 - Se incrementa pwm_counter (0–255 cíclico).
4. En todo momento, **pwm_out** se calcula en combinacional:
 - Si $pwm_counter < duty_cycle \rightarrow pwm_out = 1$
 - Si no $\rightarrow pwm_out = 0$

6.- RTL

The RTL-level design involves describing the system's behavior using sequential and combinational elements:

- **Counter:** a sequential component that increments on each clock cycle.
- **Comparator:** combinational logic that compares the counter value with the desired duty cycle.
- **PWM Output:** result of the comparison (high or low level).

This design allows precise control over the signal's duty cycle, leveraging the 8-bit resolution to achieve 256 possible modulation levels.

For this explanation we did a “**pseudocode representation**” that show, how it is works.

Pseudocode representation:

```

if (counter < duty_cycle)
    pwm_out = 1;
else
    pwm_out = 0;

```

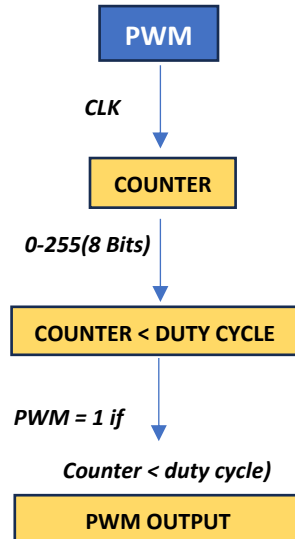


Figure: 1 PWM Function Diagram

PWM waveform representation in 16 units a brief for 256. Each bit represents a step by the counter output is "1" if counter condition is "counter < duty cycle"

Duty	0	0%	0000000000000000
Duty	4	25%	1111000000000000
Duty	8	50%	1111111100000000
Duty	12	75%	1111111111110000
Duty	16	100%	1111111111111111

5.- GENERAL DESCRIPTION

The system consists of a PWM signal generator with:

- 8-bit resolution (256 duty cycle levels).
- Fixed output frequency: 500 Hz.
- Developed in System Verilog, simulated in Vivado.

An 8-bit counter is implemented, which counts from 0 to 255 and is compared against an input value called duty_cycle. If the counter is less than duty_cycle, the output pwm_out remains high (1); otherwise, it goes low (0).

8-bit Counter: Counts from 0 to 255. Resets to 0 when it reaches the maximum. Increments on each clock cycle.

Comparator (counter < duty_cycle): Compares the counter value against the duty_cycle. If the condition is true, pwm_out = 1; otherwise, pwm_out = 0.

Control Logic: The reset forces the counter to zero. The duty_cycle value can change dynamically.

Operation Sequence: At the start, cnt = 0 (active reset). On each clock edge:

- If reset_n == 0, cnt = 0.
- If reset_n == 1, cnt = cnt + 1 (with wrap-around).

The pwm_out output changes each cycle according to cnt < duty_in. The complete PWM cycle spans 256 clock cycles → 500 Hz.

6.- PINOUT

Signal	Direction	Width	Description
Clk	In	1	System Clock (50 MHz)
Reset	In	1	Asynchronous active-low reset
duty [8..0]	In	8	Duty cycle value (0-255)
Pwm out	out	1	PWM Output

7.- DECISION & JUSTIFICATION

8-bit Counter	Provides 256 resolution levels (0–255). Compact memory and simple combinational logic. Matches the 256 possible PWM levels (8-bit resolution).
----------------------	---

Comparator	Generates the signal directly without additional logic, minimizing delay. Minimizes additional logic and reduces RTL complexity.
Synchronous output with clk.	Simulation allows assuming a single cycle per increment. Ensures stability in simulation and precise control.
Asynchronous reset (reset_n).	Ensures a defined initial state: cnt = 0, pwm_out = 0. Facilitates deterministic behavior in simulation.

8.- CODE

For the **RTL design**, we took into account the creation of a **module containing the logic signals required for system clock handling and PWM duty cycle control**.

```
module pwm_8bit_500hz #(
    parameter CLK_FREQ_HZ = 50_000_000 // Frecuencia del reloj del sistema e
)(
    input logic clk, // Señal Del Reloj del Sistema
    input logic rst, // Señal De Reinicio Sincronico (Acti
    input logic [7:0] duty_cycle, // Señal ciclo de trabajo de PWM (0-2
    output logic pwm_out // Señal De Salida del PWM
);
```

In this section, we define all local parameters related to counters, widths, steps, and frequencies as variables.

```
// Aquí se define el numero maximo de pasos del contador PWM (resolucion de 8-bit = 256 niveles)
localparam int PWM_RESOLUTION = 256;

// Aquí se calcula cuantos ciclos de reloj se requieren por cada paso del contador PWM
// Esto asegura la frecuencia del PWM deseada
localparam int PWM_TARGET_FREQ = 500;
localparam int TICKS_PER_STEP = CLK_FREQ_HZ / (PWM_TARGET_FREQ * PWM_RESOLUTION);

// Tamaño De Ancho De Bit Necesario para contar hasta TICKS_PER_STEP
localparam int TICK_COUNTER_WIDTH = $clog2(TICKS_PER_STEP);

// Contador que acumula los ciclos de reloj hasta alcanzar un paso del PWM
logic [TICK_COUNTER_WIDTH-1:0] tick_counter;

// Contador de 8 bits que determina el valor actual del PWM (0-255)
logic [7:0] pwm_counter;
```

In this section, we implement the sequential logic that handles tick counting and increments the PWM counter.

```
// Logica Secuencial Principal (Maneja conteo de ticks e incremento contador PWM)
always_ff @(posedge clk or posedge rst) begin
    if (rst) begin
        // Si se activa "reset", se reinician ambos contadores
        tick_counter <= 0;
        pwm_counter <= 0;
    end else begin
        if (tick_counter == TICKS_PER_STEP - 1) begin
            // Cuando se alcanza el numero de ticks por paso, se reinicia el "contador de ticks"
            // tambien se incrementa el contador PWM
            tick_counter <= 0;
            pwm_counter <= pwm_counter + 1;
        end else begin
            // Se incrementa tick_counter hasta alcanzar TICKS_PER_STEP (si aun no llega, sigue contando ticks)
            tick_counter <= tick_counter + 1;
        end
    end
end
```

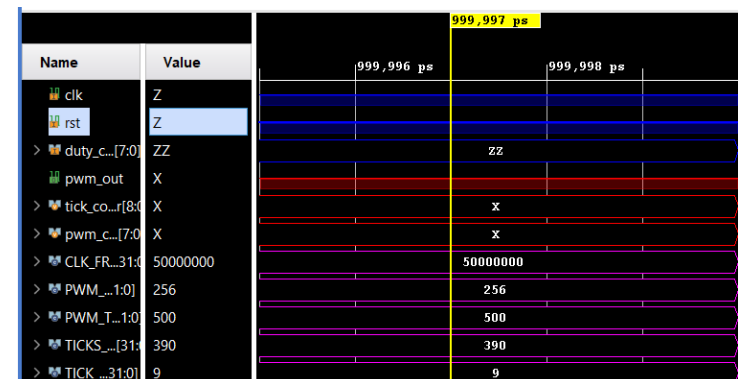
This section includes the combinational logic that performs the comparison to generate the PWM output.

```
// Logica Combinacional De Salida (Se genera la salida PWM comparando el "contador" con el "duty_cycle")
always_comb begin
    // Si el valor "contador" es menor al ciclo de trabajo, la salida es alta
    // de lo contrario la salida es baja
    pwm_out = (pwm_counter < duty_cycle);
end

endmodule
```

9.- WAVEFORM

In this case we saw this in the “Run Simulation” in the tool “Vivado”.



Also was possible saw a representation of the schematic RTL, but in this case for the “Team13” did the Design Verification for this RTL stage.

